

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	SIVARANGJENII B	Reg. No.:	192521375
Section / Batch:	B.TECH IT	Date:	25.8.26

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system continuously receives large blocks of data from a high-speed I/O device. Write pseudocode to design a DMA-based data transfer algorithm that transfers data directly from the I/O device to main memory with minimal CPU involvement. The algorithm should initialize the DMA controller, configure source and destination addresses, specify the transfer size, initiate the transfer, and handle completion and error interrupts.
2. An embedded system is connected to multiple I/O devices, each capable of generating interrupts with different priority levels. Develop pseudocode for an interrupt handling mechanism that identifies and prioritizes vectored interrupts, determines the interrupting device, and dispatches the appropriate interrupt service routine. The algorithm should ensure that high-priority interrupts are serviced efficiently while maintaining proper handling of lower-priority requests.
3. A computer system communicates with different I/O devices having varying latency, bandwidth, and data transfer requirements. Write pseudocode for a dynamic I/O selection mechanism that chooses between memory-mapped I/O and I/O-mapped I/O based on the performance requirements of each device. The algorithm should evaluate latency and bandwidth requirements and select the mapping technique that provides efficient communication.
4. A computer system contains multiple devices that simultaneously request access to high-speed communication interfaces such as PCIe, USB, and Thunderbolt. Design pseudocode for a bus arbitration algorithm that manages competing access requests and allocates the appropriate interface to each device. The algorithm should consider device priority, bandwidth requirements, interface availability, and fairness to ensure efficient utilization of communication resources.
5. A real-time embedded system communicates with both low-speed devices, such as simple sensors, and high-speed devices, such as data acquisition units and storage devices. Write pseudocode for a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupt-driven communication for high-speed devices. The algorithm should identify the device type, select the appropriate I/O technique, initiate data transfer, and handle completion and error conditions while minimizing CPU involvement.

I SIVARANGJENII B(192521375) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SIVARANGJENII B

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	

	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator Faculty Incharge

Q1. DMA-Based Data Transfer Algorithm

Problem Understanding

The system continuously receives large blocks of data from a high-speed I/O device. Direct CPU-based transfer would consume considerable CPU time. Therefore, DMA (Direct Memory Access) is used so that data can be transferred directly from the I/O device to main memory.

The algorithm must initialize the DMA controller, configure the source and destination addresses, specify the transfer size, start the transfer, and handle both completion and error interrupts.

Inputs

- I/O device source address
- Main memory destination address
- Transfer size
- DMA controller
- Transfer status

Output

- Data transferred from the I/O device to main memory
- Completion notification
- Error notification if the transfer fails

Pseudocode

ALGORITHM DMA_Data_Transfer

BEGIN

INITIALIZE DMA_Controller

```
SET Source_Address = IO_Device_Address
SET Destination_Address = Main_Memory_Address
SET Transfer_Size = Required_Data_Size
CONFIGURE DMA_Controller
Source ← Source_Address
Destination ← Destination_Address
Size ← Transfer_Size
Direction ← IO_TO_MEMORY
Interrupt ← ENABLE
```

```
CLEAR DMA_Status
START DMA_Transfer\
WHILE DMA_Transfer is ACTIVE DO
CPU performs other tasks
END WHILE
END
DMA Completion Interrupt
INTERRUPT DMA_Completion_Handler
BEGIN
IF DMA_Status = TRANSFER_COMPLETE THEN
CLEAR Completion_Interrupt
VERIFY Data_Transfer
IF Data is VALID THEN
SET Transfer_Status = SUCCESS
NOTIFY CPU
ELSE
SET Transfer_Status = DATA_ERROR
NOTIFY CPU
END IF
END IF
END INTERRUPT
```

DMA Error Interrupt

INTERRUPT DMA_Error_Handler

BEGIN

READ DMA_Error_Status

STOP DMA_Transfer

IF Error_Type = INVALID_ADDRESS THEN

REPORT "Invalid Address"

ELSE IF Error_Type = TRANSFER_ERROR THEN

REPORT "Transfer Error"

ELSE

REPORT "Unknown DMA Error"

END IF

CLEAR DMA_Error

IF Retry_Allowed = TRUE THEN

REINITIALIZE DMA_Controller

RESTART DMA_Transfer

ELSE

REPORT "DMA Transfer Failed"

END IF

END INTERRUPT

Control Structures Used

The algorithm uses:

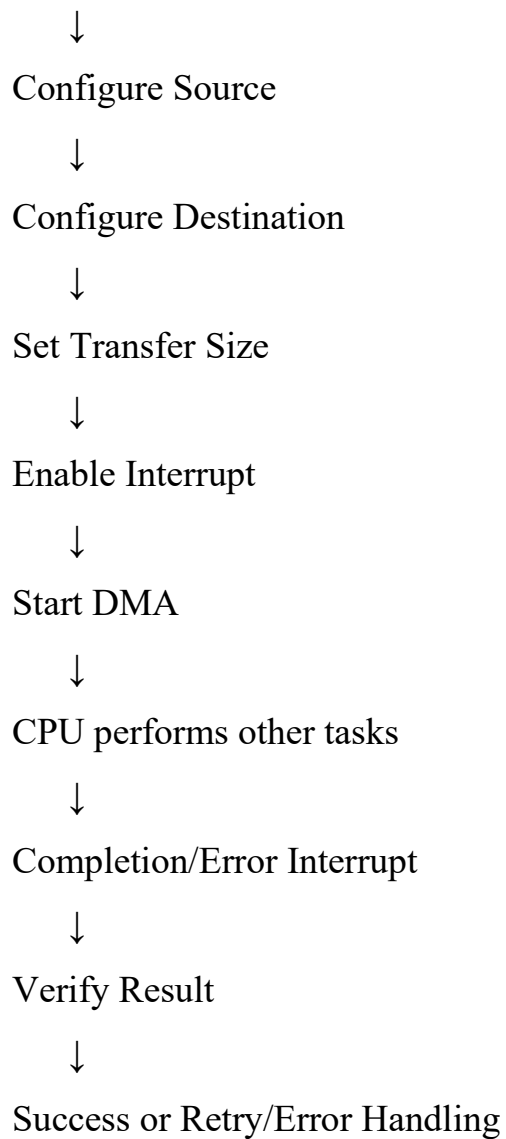
- Sequence for DMA initialization and configuration
- WHILE loop to monitor the transfer state
- IF-ELSE conditions to handle completion and errors
- Interrupt handlers for asynchronous completion and error notification

The CPU does not continuously move the data. It is free to perform other tasks while DMA performs the transfer.

Completeness

The algorithm handles the complete DMA process:

Initialize DMA



Conclusion

The DMA-based algorithm minimizes CPU involvement by allowing the DMA controller to directly transfer large data blocks from the I/O device to main memory. Interrupts are used to notify the CPU only when the transfer completes or an error occurs.

Q2. Vectored Interrupt Handling Mechanism

Problem Understanding

The embedded system contains multiple I/O devices that can generate interrupts with different priority levels. The interrupt mechanism must identify the interrupt source using its vector, prioritize simultaneous requests, and execute the appropriate interrupt service routine.

High-priority interrupts must be serviced first, while lower-priority requests must remain pending until they can be handled.

Inputs

- Interrupt requests from multiple devices
- Interrupt vector
- Device priority
- Interrupt status
- ISR address

Output

- Correct ISR execution
- High-priority interrupts serviced first
- Lower-priority interrupts retained for later processing

Pseudocode

ALGORITHM Interrupt_Handler

BEGIN

INITIALIZE Interrupt_Controller

CONFIGURE Interrupt_Vectors

CONFIGURE Device_Priorities

ENABLE Interrupts

WHILE System_Is_Running = TRUE DO

IF Interrupt_Request = TRUE THEN

Pending_List ← READ_PENDING_INTERRUPTS

Highest_Priority ← FIND_HIGHEST_PRIORITY(Pending_List)

Vector ← GET_INTERRUPT_VECTOR(Highest_Priority)

Device ← IDENTIFY_DEVICE(Vector)

CALL Dispatch_ISR(Device, Vector)

END IF

END WHILE

END

Interrupt Dispatch Procedure

PROCEDURE Dispatch_ISR(Device, Vector)

BEGIN

SAVE CPU_Context

DISABLE Lower_Priority_Interrupts

IF Device = HIGH_PRIORITY_DEVICE THEN

ENABLE Higher_Priority_Nesting

END IF

ISR_Address ← GET_ISR_ADDRESS(Vector)

JUMP TO ISR_Address

END PROCEDURE

Interrupt Service Routine

ISR(Device)

BEGIN

READ Device_Status

IF Device_Status = DATA_AVAILABLE THEN

READ Device_Data

STORE Data

ELSE IF Device_Status = ERROR THEN

HANDLE Device_Error

END IF

CLEAR Device_Interrupt

RESTORE CPU_Context

RETURN FROM INTERRUPT

END ISR

Priority Handling

The interrupt controller maintains the priority of each device.

Highest Priority



Critical Device



High Priority Device



Normal Priority Device



Low Priority Device



Background Device

If multiple devices request service simultaneously, the highest-priority request is selected first.

Lower-priority requests remain in the pending queue.

Preventing Starvation

To ensure that lower-priority devices are not ignored indefinitely, priority aging can be used.

IF Device_Wait_Time > Maximum_Wait_Time THEN

 INCREASE Device_Priority

END IF

This allows a continuously waiting low-priority request to eventually receive service.

Control Structures Used

The algorithm uses:

- WHILE loop for continuous interrupt monitoring
- IF-ELSE conditions for device status and priority
- Function/procedure calls for vector lookup and ISR dispatch
- Priority selection for simultaneous interrupts

Completeness

The complete operation is:

Interrupt Request



Read Pending Interrupts



Determine Priority



Obtain Interrupt Vector



Identify Device



Locate ISR



Execute ISR



Clear Interrupt



Restore CPU Context



Service Next Pending Interrupt

Conclusion

The proposed mechanism efficiently handles multiple vectored interrupts by identifying the interrupt source through its vector and selecting the highest-priority pending request. Lower-priority requests remain pending and can use priority aging to prevent starvation.

Q3. Dynamic I/O Selection Mechanism

Problem Understanding

Different I/O devices have different latency, bandwidth and data-transfer requirements.

Therefore, one I/O mapping technique may not be optimal for every device.

The algorithm must evaluate the device's performance requirements and dynamically choose between memory-mapped I/O (MMIO) and I/O-mapped I/O.

Inputs

- Device type
- Latency requirement
- Bandwidth requirement
- Data transfer size
- Device register requirements

Output

- Selected I/O mapping technique
- Configured device communication method

Selection Logic

Memory-mapped I/O is generally suitable when:

- Low-latency register access is important
- High-frequency register access is required
- The processor provides efficient memory access instructions
- The device requires large or frequent data transfers

I/O-mapped I/O can be selected when:

- The processor provides efficient dedicated I/O instructions
- The device has a small register set
- The device does not require frequent memory-style access

Pseudocode

ALGORITHM Dynamic_IO_Selection

BEGIN

READ Device_Type

READ Latency_Requirement

READ Bandwidth_Requirement

READ Transfer_Size

IF Latency_Requirement = HIGH THEN

SELECT Memory_Mapped_IO

ELSE IF Bandwidth_Requirement = HIGH THEN

SELECT Memory_Mapped_IO

ELSE IF Transfer_Size = LARGE THEN

SELECT Memory_Mapped_IO

ELSE

SELECT I/O_Mapped_IO

END IF

CONFIGURE Selected_IO_Method

INITIALIZE Device

RETURN Selected_IO_Method

END

Performance Evaluation

A more detailed decision can be made using a performance score.

IF Latency_Requirement = HIGH AND

Bandwidth_Requirement = HIGH THEN

Mapping $\leftarrow$ MEMORY_MAPPED_IO

ELSE IF Latency_Requirement = HIGH THEN

Mapping $\leftarrow$ MEMORY_MAPPED_IO

ELSE IF Bandwidth_Requirement = HIGH THEN

Mapping $\leftarrow$ MEMORY_MAPPED_IO

ELSE

Mapping $\leftarrow$ IO_MAPPED_IO

END IF

Example Decision

High-Speed Data Acquisition Device

↓

High Bandwidth

↓

Memory-Mapped I/O

Simple Low-Speed Sensor

↓

Low Bandwidth

↓

I/O-Mapped I/O

Control Structures Used

The algorithm uses:

- IF-ELSE conditions to evaluate performance requirements
- Logical AND conditions to evaluate multiple requirements
- Procedure calls for device configuration

Completeness

The algorithm considers:

- Latency
- Bandwidth
- Transfer size
- Device requirements
- Mapping selection
- Device initialization

This makes the mechanism adaptable to different types of I/O devices.

Justification

Memory-mapped I/O is selected for devices requiring fast and frequent register access

because it allows device registers to be accessed through the processor's normal memory-access mechanism.

I/O-mapped I/O can be selected for simpler devices where high bandwidth and very low latency are not essential.

Conclusion

The dynamic selection mechanism evaluates the requirements of each device before selecting the I/O mapping technique. This avoids using a single method for all devices and allows the system to achieve efficient communication based on actual performance requirements.

Q4. Bus Arbitration Algorithm

Problem Understanding

Multiple devices simultaneously request access to high-speed communication interfaces such as PCIe, USB and Thunderbolt. Since several devices may request resources at the same time, the system requires a bus arbitration mechanism.

The algorithm must consider:

- Device priority
- Bandwidth requirement
- Interface availability
- Fairness

The objective is to allocate the most appropriate interface while preventing one device from continuously occupying the available resources.

Inputs

- Device requests
- Device priority
- Required bandwidth
- Interface availability
- Current interface allocation
- Waiting time

Output

- Selected device
- Allocated communication interface
- Fair access to communication resources

Pseudocode

ALGORITHM Bus_Arbitration

BEGIN

INITIALIZE PCIe_Interface

INITIALIZE USB_Interface

```

INITIALIZE Thunderbolt_Interface
WHILE System_Is_Running = TRUE DO
Request_List ← GET_PENDING_REQUESTS

IF Request_List is NOT EMPTY THEN
FOR EACH Request IN Request_List DO
READ Device_Priority
READ Bandwidth_Requirement
READ Interface_Requirement
READ Waiting_Time
IF Waiting_Time > Maximum_Wait_Time THEN
INCREASE Request_Priority
END IF
END FOR
Selected_Request ← SELECT_HIGHEST_PRIORITY(Request_List)
Interface ← FIND_AVAILABLE_INTERFACE(
Selected_Request,
Bandwidth_Requirement
)
IF Interface IS AVAILABLE THEN
ALLOCATE Interface
START Data_Transfer
ELSE
PLACE Request IN Waiting_Queue

END IF
END IF
END WHILE
END

Interface Selection
FUNCTION Select_Interface(Request)

```


BEGIN

IF Request requires very high bandwidth THEN

IF Thunderbolt is AVAILABLE THEN

RETURN Thunderbolt

ELSE IF PCIe is AVAILABLE THEN

RETURN PCIe

END IF

ELSE IF Request requires high bandwidth THEN

IF PCIe is AVAILABLE THEN

RETURN PCIe

ELSE IF Thunderbolt is AVAILABLE THEN

RETURN Thunderbolt

END IF

ELSE

IF USB is AVAILABLE THEN

RETURN USB

ELSE

RETURN ANY_AVAILABLE_INTERFACE

END IF

END IF

END FUNCTION

Fairness Mechanism

A device that has waited for a long time should gradually receive higher priority.

FOR EACH Waiting_Request

Waiting_Time $\leftarrow$ Waiting_Time + 1

IF Waiting_Time > Threshold THEN

Priority $\leftarrow$ Priority + Aging_Factor

END IF

END FOR

This prevents starvation.

Bandwidth Consideration

The arbitration algorithm should not allocate an interface to a device if the available bandwidth is insufficient.

IF Available_Bandwidth $\geq$ Required_Bandwidth THEN

ALLOCATE Interface

ELSE

SEARCH FOR ANOTHER INTERFACE

END IF

Control Structures Used

The algorithm uses:

- WHILE loop for continuous arbitration
- FOR loop for examining pending requests
- IF-ELSE conditions for priority and interface selection
- Functions for interface selection
- Waiting queues for unavailable resources

Complete Arbitration Flow

Device Request



Check Priority



Check Waiting Time



Check Bandwidth Requirement



Check Interface Availability



Select Suitable Interface



Allocate Interface



Start Transfer



Release Interface



Serve Next Request

Justification

Priority ensures that important devices receive timely service. Bandwidth evaluation ensures that the selected interface can support the requested transfer.

Interface availability prevents conflicts, while priority aging prevents starvation and improves fairness.

Therefore, the algorithm provides efficient utilization of PCIe, USB and Thunderbolt communication resources.

Conclusion

The proposed bus arbitration algorithm dynamically allocates communication interfaces based on priority, bandwidth, availability and fairness. It ensures efficient resource utilization while preventing any device from being permanently denied access.

Q5. Hybrid I/O Communication Mechanism

Problem Understanding

The system communicates with both low-speed and high-speed devices. Low-speed devices such as simple sensors do not generate enough data to justify DMA, while high-speed devices such as data acquisition units and storage devices generate large amounts of data.

Therefore, the system should use polling for low-speed devices and DMA with interrupt-driven communication for high-speed devices.

Inputs

- Device type
- Device speed
- Data rate
- Transfer size
- Device status

Output

- Selected I/O technique
- Successful data transfer
- Completion/error handling

Device Classification

Device

↓

Check Data Rate

↓

Low Speed

High Speed

|

|

↓

↓

Polling

DMA

|

|

↓

↓

CPU checks

Interrupt

status

notification

Pseudocode

ALGORITHM Hybrid_IO_Communication

BEGIN

INITIALIZE IO_Devices

FOR EACH Device IN IO_Devices DO

READ Device_Type

READ Data_Rate

IF Data_Rate $\leq$ LOW_SPEED_THRESHOLD THEN

Device_Mode $\leftarrow$ POLLING

ELSE

Device_Mode $\leftarrow$ DMA_INTERRUPT

END IF

IF Device_Mode = POLLING THEN

CALL Poll_Low_Speed_Device(Device)

ELSE

CALL Configure_DMA(Device)

END IF

END FOR

END

Polling Mechanism for Low-Speed Devices

PROCEDURE Poll_Low_Speed_Device(Device)

BEGIN

WHILE System_Is_Running = TRUE DO

READ Device_Status

IF Device_Status = DATA_AVAILABLE THEN

READ Device_Data

PROCESS Device_Data

```
ELSE IF Device_Status = ERROR THEN
HANDLE Device_Error
END IF
WAIT Polling_Interval
END WHILE
END PROCEDURE
```

The polling interval prevents the CPU from continuously checking the device without any delay.

DMA Mechanism for High-Speed Devices

PROCEDURE Configure_DMA(Device)

```
BEGIN
SET Source_Address = Device_Address
SET Destination_Address = Main_Memory_Address
SET Transfer_Size = Required_Size
INITIALIZE DMA_Controller
CONFIGURE DMA_Controller
ENABLE DMA_Completion_Interrupt
ENABLE DMA_Error_Interrupt
START DMA_Transfer
END PROCEDURE
```

DMA Completion Handler

INTERRUPT DMA_Completion_Handler

```
BEGIN
IF DMA_Status = COMPLETE THEN
CLEAR DMA_Interrupt
VERIFY Data
IF Data = VALID THEN
SET Transfer_Status = SUCCESS
NOTIFY Processing_Task
ELSE
SET Transfer_Status = DATA_ERROR
```

END IF

END IF

END INTERRUPT

DMA Error Handler

INTERRUPT DMA_Error_Handler

BEGIN

READ Error_Status

STOP DMA_Transfer

RECORD Error

IF Retry_Count < MAX_RETRIES THEN

Retry_Count $\leftarrow$ Retry_Count + 1

RESTART DMA_Transfer

ELSE

REPORT "DMA Transfer Failed"

END IF

END INTERRUPT

Device Selection Logic

IF Device_Type = LOW_SPEED_SENSOR THEN

USE POLLING

ELSE IF Device_Type = DATA_ACQUISITION_UNIT OR

Device_Type = STORAGE_DEVICE THEN

USE DMA_WITH_INTERRUPTS

END IF

Control Structures Used

The algorithm uses:

- FOR loop to examine all connected devices
- WHILE loop for continuous polling
- IF-ELSE conditions to classify devices
- Procedures for polling and DMA configuration
- Interrupt handlers for DMA completion and errors
- Retry condition for failed transfers

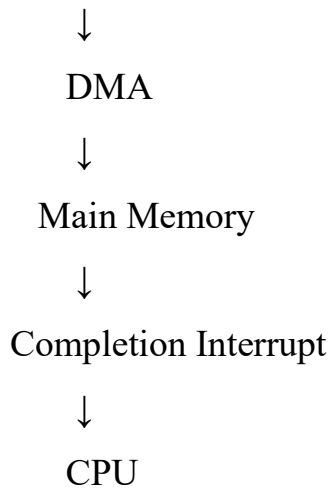
CPU Involvement Analysis

For low-speed devices, polling is acceptable because the data rate is low and the polling interval can be controlled.

For high-speed devices, polling would cause unnecessary CPU usage because the CPU would have to repeatedly check the device status.

DMA removes the CPU from the bulk data-transfer path:

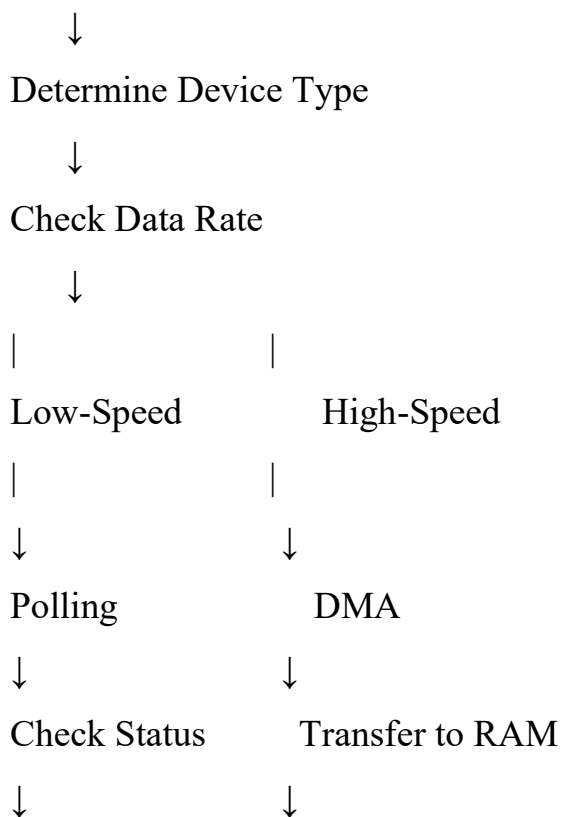
High-Speed Device

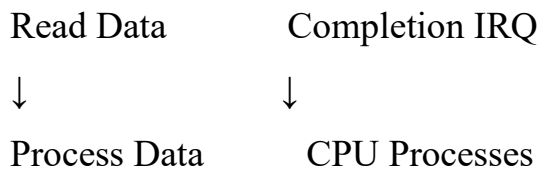


The CPU only performs configuration and handles completion/error notifications.

Complete Hybrid I/O Flow

Detect Device





Justification

Polling is simple and appropriate for low-speed devices because their data rate is small and frequent interrupt or DMA operations would add unnecessary complexity.

DMA with interrupt-driven communication is more appropriate for high-speed devices because it transfers large amounts of data directly to memory without requiring the CPU to move every data item.

The hybrid mechanism therefore provides a balance between simplicity for low-speed devices and efficiency for high-speed devices.

Constraint Satisfaction

Constraint	Proposed Solution
Low-speed sensors	Polling
High-speed data acquisition	DMA
High-speed storage	DMA
Minimal CPU involvement	DMA performs bulk transfers
Transfer completion	DMA completion interrupt
Error handling	DMA error interrupt and retry
Device identification	Device type/data-rate check

Conclusion

The proposed hybrid I/O mechanism dynamically selects the appropriate communication method according to device speed and data requirements. Low-speed devices use controlled polling, while high-speed devices use DMA with interrupt-driven completion and error handling. This minimizes CPU involvement while maintaining efficient communication with all connected devices.